Codexify Schema Review Snapshot – For Claude

This document serves as a technical snapshot for Claude, a technical contributor at Resonant Constructs LLC, to review and audit the current state of the Codexify schema architecture. Claude has been tasked with improving scalability, maintainability, and schema cohesion across Postgres, SQLite, and Neo4j layers. This document outlines the current implementation realities, architectural disconnects, and future directions for schema unification. It is based on live collaboration and audit sessions between Codexify's builder (Chris) and Claude.

Summary of Schema Layers:
• **SQLAlchemy ORM Models:** Defined in *guardian/db/models.py*, used by Alembic for declarative migrations.
• **Imperative SQL Layer:** Defined in *guardian/core/db.py* (GuardianDB), uses raw SQL to manage legacy tables like *chat_threads, chat_messages, connector_configs*, etc.
• **Neo4j Graph Layer:** Separate initialization logic with minimal schema enforcement.
• **Alembic Status:** Migration system is active, but baseline files are empty; most table creation is delegated to raw SQL in app code.
• **Seeding:** Managed via *seed_defaults.py* script which has now been patched for idempotency.

Message to Claude:
We've made progress unifying our seed process and bootstrapping infrastructure (including Docker Compose and migrator service flow), but we are still living in dual schema land (SQLAlchemy + raw SQL). Please advise on migration strategy for moving raw schema definitions from GuardianDB into Alembic-managed declarative form *(if warranted)*. Would love insight into hybrid strategies or safety-first transitional patterns. Use this doc as grounding to refine further steps. You are empowered to make recommendations that improve reliability and developer confidence. Thank you!
– Chris (Builder, Resonant Constructs)

Generated on 2025-10-26 16:57 UTC